

V. МЕНЮ. ДИНАМИЧНИ КОНТРОЛИ. ТЕКСТОВИ ФАЙЛОВЕ. ВАЛИДИРАНЕ НА ДАННИ

1. Работа с менюта

За реализиране на менюта в средата Visual C# се използват компонентите MenuStrip и ContextMenuStrip.

1.1. Създаване на лента с инструменти във форма на приложение

За реализиране на ленти с инструменти в средата Visual C# се използва компонента ToolStrip.

1.2. Създаване на лента за състояние във форма на приложение

За реализиране на лента за състоянието в средата Visual C# се използва компонента StatusStrip.

1.3. Диалози за избор на файл

Windows Forms предоставя стандартни диалози, които осигуряват достъп до стандартно настроените диалогови прозорци, извършващи операции по отваряне и съхранение на файлове (прозорци Open и Save As) т.е. компонентите OpenFileDialog и SaveFileDialog.

1.4. Диалози за избор на шрифт и цвят.

Компонентите са FontDialog и ColorDialog.

2. Създаване на приложения чрез използване на динамични контроли

Добавяне на контроли в Windows Form Application може да стане по два начина чрез издърпване на контролата от меню Toolbox и динамично, чрез програмен код. Разликата между двата подхода е, че в първият контролата съществува физически в проекта, докато при втория, тя се визуализира едва след стартиране на приложението.

Задача1. *Да се създаде приложение Динамични контроли. Да се добави button1, да се създаде метод button1_Click(object sender, EventArgs e). При кликуване върху button1 всеки път да се създава динамично по един textBox.*

```
using System;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace DynamicBuuton
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        int cLeft = 1;

        public System.Windows.Forms.Button AddNewButton()
        {
            System.Windows.Forms.Button button = new System.Windows.Forms.Button();
            this.Controls.Add(button);
            //button.Top = cLeft;
            //button.Left = 0;
            button.Size = new System.Drawing.Size(120, 30);
            button.Location = new System.Drawing.Point(0, 0);
            button.Text = "Button" + this.cLeft.ToString();
            cLeft = cLeft + 1;
            button.Click += new EventHandler(this.button_Click);
        }
    }
}
```

```

        return button;
    }

    private void Form1_Load(object sender, EventArgs e)
    {
        AddNewButton();
    }

    public System.Windows.Forms.TextBox AddNewTextBox()
    {
        System.Windows.Forms.TextBox txt = new System.Windows.Forms.TextBox();
        this.Controls.Add(txt);
        txt.Top = cLeft * 25;
        txt.Left = 100;
        txt.Text = "TextBox " + this.cLeft.ToString();
        cLeft = cLeft + 1;
        return txt;
    }

    private void button_Click(object sender, EventArgs e)
    {
        AddNewTextBox();
    }
}

```

3. Използване на регулярни изрази

3.1. Предимства

Регулярните изрази са мощно, удобно и ефективно средство за обработка на текст. Чрез тях се:

- Търси и извлича информация от даден текст по даден шаблон;
- Валидира текстова информация;
- Заменя и изтрива поднизове в даден текст чрез шаблони.

Те използват теорията на крайните автомати и регулярните езици. В .NET Framework регулярните изрази имат синтаксиса на Perl 5.

3.2. Определение

Регулярен израз е символен низ, който описва някаква съвкупност от символни низове (регулярен език). Регулярните изрази използват специална граматика за описание на низовете.

Например регулярният израз обозначава всички: непразни низове, които се състоят само от цифрите 0 и 1, а низът телефонни номера, които имат вида 088XXXXXXX (X е цифра)

3.3. Езикът на регулярните изрази

Регулярните изрази са средство за описание на поднизове, които търсим в даден текст.

3.4. Синтакс

Синтаксисът на регулярните изрази се състои от литерали и метасимволи:

- литералите са части от низове, които търсим;
- метасимволите са команди, които задават специални указания и правила за търсене.
- в регулярния израз: `w*аз|съм|w*програмистw`, литерали са "аз", "съм" и "програмист", а метасимволи са "`w`", "`*`" и "`|`"

Метасимволите в регулярните изрази са няколко категории:

- Escaping последователности, например `\*`
- Класове от символи, например `[a-zA-Z]`
- Метасимволи за задаване на количеството (quantifiers), например `*` и `+`
- Метасимволи за задаване на местоположението в текста, например `\b`
- Метасимволи за алтернативен избор, например `|` (логическо "или")
- Групиращи метасимволи, например `([0-9]+)`
- Други метасимволи, например `#` (за задаване на коментари), заместващи (например `$1`)

и други.

Тези категории са описани в Таблица 5.1.

Таблица 5.1.

Escaping последователности	
\t	табулация
\r	символ за връщане на каретката CR (0x0D)
\n	символ за нов ред LF (0x0A)
\xxx	символ с ASCII код xx (шестнайсетично)
\uXXXX	Unicode символ с номер XXXX (шестнайсетично)
\\	символ \
*	символ *
\+	символ +
Класове от символи	
.	– обозначава произволен символ без \n
(. \s)	– обозначава произволен символ
[символи]	– обозначава произволен символ от изброените
Пример: [01]	обозначава цифрата 0 или 1
[^символи]	– обозначава произволен символ, който не е сред изброените
Пример: [^<>\\]	обозначава всеки символ без <, > и \
[charX-charY]	– обозначава символ в зададения интервал
Пример: [0-9A-F]	обозначава всеки символ, който е цифра или латинска буква между A и F
\w	– обозначава буквите, цифрите и символа _ (за всички езици от Unicode)
\W	– обозначава всички символи с изключение на буквите, цифрите и _
\s	– обозначава символите за празно пространство (интервал, табулация, нов ред, ...)
\S	– обозначава символите, които не са празно пространство
\d	– обозначава десетичните цифри [0-9]
\D	– обозначава всички символи, които не са десетични цифри
Метасимволи за количество	
*	– нула или повече срещания
Пример: [01]*	обозначава всички символни низове, съставени от цифрите 0 или 1, включително празния низ
+	– едно или повече срещания
Пример: [A-Z]+	задава непразните символни низове, съставени от главни латински букви
?	– нула или едно срещания
Пример: [A-Z]?	обозначава главна латинска буква или празен низ
{n}	– точно n срещания
Пример: [0-9]{3}	обозначава последователност от точно 3 цифри
{n,}	– поне n срещания
Пример: [0-9]{5,}	обозначава последователност от поне 5 цифри
{n,m}	– поне n и най-много m срещания
Пример: [0-9]{2,4}	обозначава последователност от 2, 3 или 4 цифри
*?	– нула или повече срещания, но най-малкият възможен брой
+?	– едно или повече срещания, но най-малкият възможен брой
{n,}?	– поне n срещания, но най-малкият възможен брой
Метасимволи за местоположение	
\b	– указание за търсене само в началото или края на дума – на границата между символите \w и \W, но в рамките на \w
Пример: \b\w*мир\b	обозначава всички думи, съдържащи като подниз "мир", например "мирът", "омиротворявам" но не и "няма да има примирие"
Пример: .*?ra\b	обозначава всички най-къси поднизове, завършващи на "ла", например "Кала " и "има игла", но не и "лалето не цъфти " и "посади лалета"
\B	– указание за търсене само в средата на думата (без началото и края)
Пример: \w*игла\B	ще намери "игла" в текста "Къде ми е иглата?", но няма да намери нищо в текста "Подайте ми иглата, моля!"
\A	– указание за търсене само в началото на подадения текст (задава се преди низа)
Пример: \Aсвобода	ще намери "свобода" в текста "свободата е всичко", но няма да намери нищо в текста "Без свобода нямаш развитие!"
\Z	– указание за търсене само в края на текста (задава се след низа)
Пример: домакиня\Z	ще намери "домакиня" в текста "добра домакиня", но няма да намери нищо в текста "днес домакинята си почива"
^	– търсене само в началото на текста (в режим multi-line и в началото на всеки ред)
\$	– търсене само в края на текста (в режим multi-line и в края на всеки ред)
Метасимволи за избор	
A B	– задава алтернативен избор между регулярните изрази A и B
Пример: жена мъж	ще намери "жена" в текста "Жена ми е на пазар.", а в текста "Мъж ли е това?" ще намери "мъж"
Метасимволи за задаване на групи	
(група), (?<име>група)	– задава група в регулярния израз (без име или с име)
Пример: \s*(?<name>\w+)\s*	символ за име (d+)

Групите се използват за логическо отделяне на части от регулярния израз и могат да имат имена.

В .NET Framework средствата за обработка на текст чрез регулярни изрази са разположени в пространството от имена System.Text.RegularExpressions.

Класът **Regex** съдържа неизменен регулярен израз и методи за търсене, заместване и разделяне на низове чрез този израз. **Regex** предоставя и статични методи за всички основни операции.

Класът **Match** съдържа описание на едно съвпадение (стойност, начална позиция и дължина), получено в резултат от търсене с регулярен израз. Дава възможност за намиране на следващото съвпадение от търсенето, ако има такова.

Класът **MatchCollection** съдържа списък от съвпадения (получени в резултат от търсене).

Класът **Group** представлява група от символи, съдържаща се в дадено съвпадение (**Match**). В едно съвпадение може да има няколко групи.

Класът **GroupCollection** съдържа списък от групи, съдържащи се в дадено съвпадение.

Делегатът **MatchEvaluator** се използва при заместване с регулярен израз за обработка на всяко едно съвпадение.

Изброеният тип **RegexOptions** се използва за задаване на опции за търсенето с регулярни изрази. В **.NET Framework** класът **Regex** е най-важният клас за работа с регулярни изрази. С **Regex** може да се работи по два начина:

- Инстанцира се класът и в конструктора му се подава регулярен израз, след което се извикват методите за обработка на текст (**IsMatch**, **Match**, **Matches**, **Replace**, **Split**)

- Използват се статичните методи на класа (**IsMatch**, **Match**, **Matches**, **Replace**, **Split**), на които се подава текста за обработка и регулярен израз, който да се използва

Подходът с инстанцирането на **Regex** е по-ефективен ако с един и същ израз ще се обработват последователно няколко текста.

По-важни методи и свойства на **Regex** са:

IsMatch(text, pattern) – проверява дали в даден текст се среща поне един подниз, който съответства на даден регулярен израз.

Match(text, pattern) – търси зададения регулярен израз в зададения текст и връща първото съвпадение като **Match** обект.

Matches(text, pattern) – търси зададения регулярен израз в зададения текст и връща **MatchCollection** от всички съвпадения.

Replace(text, pattern, replacement) – замества всички срещания за даден регулярен израз в даден текст със заместващ текст, който може да съдържа части от намерените съвпадения (групи в рег. израз).

string[] Split(text, pattern) – разделя даден низ на части по даден регулярен израз.

Escape(text) – замества всички специални символи за езика на регулярните изрази с еквивалентни **escaping** последователности.

Unescape(text) – обратното на **Escape**().

GetGroupNames() – връща масив от имената на групите, дефинирани в даден регулярен израз (групите без имена автоматично получават служебно име).

Options – свойство от тип **RegexOptions**, което задава някои настройки на изрази (като **Singleline**, **Multiline**, **IgnoreCase** и **IgnorePatternWhitespace**).

4. Работа с текстови файлове

4.1. Потоци

Потоците (**streams**) са важна част от всяка входно-изходна библиотека. Те намират приложение, когато програмата трябва да "прочете" или "запише" данни от или във източник на данни – файл, други компютри, сървъри и т.н.

Важно е да се уточни, че терминът вход (**input**) се асоциира с четене на информация, а терминът изход (**output**) – със записване на информация.

Логистиката на потока има за цел да скрива хардуера от погледа на програмиста и като негов символ да предоставя много проста и лесна за управление абстракция - поток.

Входно/Изходната система (**IOS**) на **C#** се основава на идеята за изграждане на абстрактно ниво на връзката между програмиста и обслужващия хардуер. В основата на тази връзка стои понятието „поток“.

4.2. Определение

Потокът е наредена последователност от байтове, които се изпращат от едно приложение или входно устройство и се получават в друго приложение или изходно устройство. Тези

байтове се изпращат и получават един след друг и винаги пристигат в същия ред, в който са били изпратени.

За да се проче или запише нещо от или във файл, трябва да се:

- отвори (създаде) потока към дадения файл;
- чете или пише в отворения поток;
- затвори потока.

4.3. Основни операции с потоци

• Създаване

Свързване на потока с източник на данни, с механизъм за пренос на данни или с друг поток.

• Четене

Извличане данни от потока. Четенето винаги се извършва последователно от текущата позиция на потока. Четенето е блокираща операция и при нея може да се получи забавяне или отсрещната страна може изобщо да не прати данни.

• Запис

Изпращане данни в потока. Записът се извършва от текущата позиция на потока и потенциално може да е блокираща операция и да се забави докато данните поемат по своя път.

• Позициониране

Преместване на текущата позиция на потока. Извършва се в потоци, които поддържат позициониране. Файловите потоци обикновено поддържат позициониране, докато мрежовите - не.

• Затваряне

Приключване работата с потока и освобождаване ресурсите, заети от него.

В .NET Framework класовете за работа с потоци се намират в пространството от имена System.IO. Разграничават се два основни типа потоци: такива, които работят с двоични данни и такива, които работят с текстови данни.

4.4. Потоци в .NET – основни класове

Двоични потоци

Работят с двоични (бинарни) данни, което ги прави универсални и могат да се използват за четене на информация от всякакви файлове. Основните класове, които се използват, за четене и запис от и към двоични потоци са: FileStream, BinaryReader и BinaryWriter.

4.5. Текстови потоци

Текстовите потоци са много подобни на двоичните, но работят само с текстови данни или по-точно с поредици от символи (char) и стрингове (string). Идеални са за работа с текстови файлове, което ги прави неизползваеми при работа с каквито и да е бинарни файлове.

Основните класове за работа с текстови потоци са TextReader и TextWriter. Те са абстрактни класове и от тях не могат да бъдат създавани обекти. Тези класове дефинират базова функционалност за четене и запис на класовете, които ги наследяват. По важните им методи са:

- ReadLine() – чете един текстов ред и връща символен низ
- ReadToEnd() – чете всичко от потока до неговия край и връща символен низ.
- Write() – записва символен низ в потока
- WriteLine() – записва един текстов ред в потока

Символите в .NET са Unicode символи, но потоците могат да работят освен с Unicode и с други кодирания (кодировки), например стандартното за кирилицата кодиране Windows-1251.

4.6. Четене от текстов файл

Текстовите файлове предоставят идеалното решение за четене и запис на данни, които се използват често, а са твърде обемисти, за да бъдат въвеждани ръчно всеки път, когато се стартира програмата. Затова сега ще се разгледа как да се чете и записва текстови файлове с класовете от .NET Framework и езика C#.

• Четене от текстов файл

C# предоставя множество начини за четене от файлове, но класът System.IO.StreamReader предоставя най-лесния начин за четене на текстов файл, тъй като наподобява четенето от конзолата. Класът StreamReader е за четене на текстов файл

- **Отваряне на текстов файл за четене**

Създаване на обект от клас StreamReader:
`StreamReader reader = new StreamReader("test.txt");`
`// файлът "test.txt" може да бъде прочетен`
`reader.Close();`

- **Отваряне на файл със задаване на кодиране**

Кодирането може да се подаде при създаването на StreamReader обект като допълнителен втори параметър:
`StreamReader reader =`
`new StreamReader("test.txt", Encoding.GetEncoding("Windows-1251"));`
`// файлът "test.txt" може да бъде прочетен`
`reader.Close();`

Като параметри в примера са подадени име на файла, който ще бъде прочетен и обект от тип Encoding. Ако не бъде зададено специфично кодиране при отварянето на файла, се използва стандартното кодиране UTF-8. В показаният по-горе случай се използва кодиране Windows-1251, който е 8-битов (еднобайтов) набор символи, проектиран от Майкрософт за езиците, използващи кирилица като български и руски.

- **Четене на текстов файл ред по ред**

Да се създаде текстов файл "primer.txt" в Debug папката на проекта (.bin\Debug). Така той ще е в същата директория, в която се компилира приложението за четене на редове и няма да се налага да се подава пълния път до него при отварянето на файла.

```
class FileReader
{
    StreamReader reader = new StreamReader("Primer.txt");
    int lineNumber = 0;
    string line = reader.ReadLine(); // Чете първия ред от файла
    while (line != null) // Чете останалите редове на файла
    {
        lineNumber++; // брояч на текущия ред за четене
        listBox1.Items.Add("lineNumber=" + lineNumber + " line=" + line);
        line = reader.ReadLine();
    }
    reader.Close();
}
```

- **Автоматично затваряне на потока**

`using(<stream object>) { stream action scope }`

Използването на using гарантира, че след излизане от тялото й ще се извика автоматично метода Close(), дори ако възникне някакво изключение.

```
class FileReader
{
    StreamReader reader = new StreamReader("Primer.txt");
    using(reader)
    {
        int lineNumber = 0;
        string line = reader.ReadLine(); // чете първия ред от файла
        while (line != null) // чете останалите редове на файла
        {
            lineNumber++; // брояч на текущия ред за четене
            listBox1.Items.Add("lineNumber=" + lineNumber + " line=" + line);
            line = reader.ReadLine();
        }
    }
}
```

- **Кодиране на файловете**

В паметта на компютрите всичко се запазва в двоичен вид. Този процес се нарича кодиране на файлове. За текстовите файлове кодирането се състои в заместване на текстовите

символи (цифри, букви, препинателни знаци и т.н.) с точно определени поредици от числови стойности. Това може да се мисли като голяма таблица: символ - стойност (пореден номер).

Тези схеми задават правила за преобразуване на текст в поредица от байтове и обратно. Кодиращата схема е една таблица със символи и техните номера, но може да съдържа и специални правила. Най-често при работа с кирилица се ползват схемите: UTF-8 и Windows-1251.

- **Кодиращи схеми (character encodings)**

Кодирането UTF-8 е универсално и е подходящо за всякакви езици, азбуки и писмености. То е кодираща схема, при която най-често използваните символи (латинската азбука, цифри и някои специални знаци) се кодират в един байт, по-рядко използваните Unicode символи (като кирилица, гръцки и арабски) се кодират в два байта, а всички останали символи (китайски, японски и много други) се кодират в 3 или 4 байта, поддържайки по този начин всичките над 100 000 символа от Unicode стандарта.

Кодирането Windows-1251 обикновено е предназначено за текстове на кирилица (например съобщения изпратени по e-mail). То съдържа 256 символа, включващи латинската азбука, кирилицата и някои често използвани знаци.

За представяне на кодиращите схеми като обекти в .NET Framework се използва класът System.Text.Encoding, който се ползва по следния начин:

Encoding win1251 = Encoding.GetEncoding("Windows-1251");

Ако не бъде указан изрично кодиращата схема (encoding) за четене от файла, .NET Framework ще използва по подразбиране encoding UTF-8.

За да се избегнат проблеми с неправилно кодиране на файловете, трябва изрично да се задава кодирането. Иначе програмата може да работи некоректно или да се счупи на по-късен етап.

- **Стандарт Unicode**

Представлява индустриален стандарт, който позволява на електронни устройства винаги да представят и манипулират по един и същи начин текст, написан на повечето от световните писмености. Символите в C# (типовете char и string) също се представят в Unicode.

- **Писане в текстов файл**

При писането в текстови файлове се използва един клас подобен на конзолата, който се нарича StreamWriter.

- **Клас StreamWriter**

Класът StreamWriter е част от пространството от имена System.IO и се използва изключително и само за работа с текстови данни. Той наподобява класа StreamReader, но вместо методи за четене, предлага такива за записване на текст във файл. За разлика от другите потоци, преди да запише на данните на желаното място, той ги превръща в байтове.

StreamWriter дава възможност при създаването си да се определи желанието от потребителя encoding.

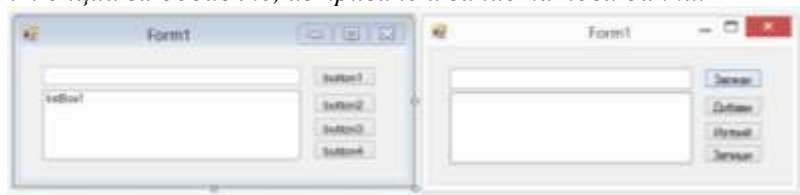
StreamWriter writer = new StreamWriter("Primer.txt");

В конструктора StreamWriter (...) на класа StreamWriter може да се подава като параметър като път до файл, така и като вече създаден поток, в който ще се записва и кодиращата схема.

5. Задачи за изпълнение:

Задачи за четене и запис от и в текстов файл:

Задача 1. *Да се създаде приложение за четене на данни от текстов файл. Да се добавят опции за добавяне, изтриване и запис на нови данни.*



Фиг.5.1. Приложение четене на данни от текстов файл

Примерно решение:

```
private void button1_Click(object sender, EventArgs e)
```

```

{
listBox1.Items.Clear();
    //зареждане на данни от текстов файл
string[] lines = System.IO.File.ReadAllLines("Primer.txt");
    foreach (string str in lines)
    { listBox1.Items.Add(str); }
}
private void button2_Click(object sender, EventArgs e)
{
    //добавяне на данни в listBox
listBox1.Items.Add(textBox1.Text);
}
private void button3_Click(object sender, EventArgs e)
{
    // премахване на данни от listBox
listBox1.Items.Remove(listBox1.SelectedItem);
}
private void button4_Click(object sender, EventArgs e)
{
    //запис на нови данни в текстов файл
string name = "Primer.txt";
StreamWriter f = File.CreateText(name);
for (int i = 0; i < listBox1.Items.Count; i++)
    f.WriteLine("{0}", listBox1.Items[i]);
f.Close();
}
private void Form1_Load(object sender, EventArgs e)
{
    button1.Text = "Зареди";
    button2.Text = "Добави";
    button3.Text = "Изтрий";
    button4.Text = "Запиши";
}

```

Варианти за добавяне на данни в текстов файл:

Вариант 1 - StreamWriter

```

TextWriter f = File.AppendText("Primer.txt");
string s = textBox1.Text;
f.WriteLine(s);
f.Close();

```

Вариант 2 - StreamWriter

```

string name = "Primer.txt";
StreamWriter f = File.AppendText(name);
string s = textBox1.Text;
f.WriteLine("{0}",s);
f.Close();

```

Четене на данни от текстов файл:

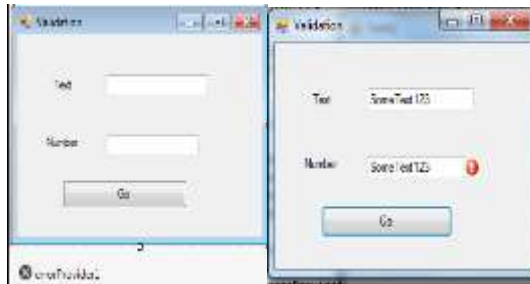
```

listBox1.Items.Clear();
string[] lines = System.IO.File.ReadAllLines("Primer.txt");
    foreach (string str in lines)
    { listBox1.Items.Add(str); }

```

Задачи за валидиране на данни:

Задача 1. Като се използва компонентата *ErrorProvider*, да се създаде приложение *Validation*, което при натискане на бутон "Go" валидира данни въведени в текстови полета.



Фиг.5.2. Интерфейс на приложение Validation (A, B), в режим изпълнение

За целта да 2се създаде форма, в която се добавят 2 текстови полета. Първото с наименование *txtString* без ограничение на допустимите символи, второто с наименование *txtNumber*, да допуска само цифри. При кликване върху бутона "Go" да се извърши проверка за запълване на текстовите полета и за съответствие на допустимите символи. В случай на грешка се задейства *SetError* метод на компонентата *ErrorProvider*.

Примерно решение:

namespace Validation2

```
{ public partial class Form1 : Form
{ public Form1()
{
InitializeComponent();
}
public static Boolean IsNumeric(string stringToTest)
{
int result;
if (int.TryParse(stringToTest, out result))
{
return true;
}
else
{
return false;
}
}
}
private void button1_Click(object sender, EventArgs e)
{ //checking if the two textboxes are cleared
if (txtString.Text == "" && txtNumber.Text == "")
{ errorProvider1.SetError(txtString, "Field must be filled up.");
errorProvider1.SetError(txtNumber, "Field must be filled up.");
}
else
{ //the error provider will be cleared.
//errorProvider1.SetError(txtString, "");
//errorProvider1.SetError(txtNumber, "");
errorProvider1.Clear();
//condition for the string value
//check if the input is a numeric value then perform the error provider
//if not, clear the error provider.
if (IsNumeric(txtString.Text))
{ //the error provider will appear and will notify the problem to the user.
errorProvider1.SetError(txtString, "Input string value is not valid.");
}
else
{ //the error provider will be cleared.
errorProvider1.SetError(txtString, "");
}
```

```

    }
    //condition for the numeric value
    //check if the input is not a numeric value then perform the error provider
    //if numeric, clear the error provider.
    if (!IsNumeric(txtNumber.Text))
    {
        //the error provider will appear and will notify the problem to the user.
        errorProvider1.SetError(txtNumber, "Input numeric value is not valid.");
    }
    else
    {
        errorProvider1.SetError(txtNumber, "");
    }
}
}
}
}
}

```

Задача 2. Да се създаде приложение *Validation1*, което да валидира три текстови полета, като използва контролата *ErrorProvider* и нейният метод *SetError()*.



Фиг.5.3. Интерфейс на приложение *Validation1* (A), в режим проектиране и изпълнение



Фиг.5.4. Интерфейс на приложение *Validation1* (B), в режим проектиране и изпълнение



Фиг.5.5. Интерфейс на приложение *Validation* (C), в режим проектиране и изпълнение

Примерно решение:

```

namespace ErrorProviderValidation
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }

        int invalidInput1 = 0; int invalidInput2 = 0; int invalidInput3 = 0;
        private void button1_Click(object sender, EventArgs e)

```

```

        { if ((invalidInput1 == 1) && (invalidInput2 == 1) && (invalidInput3 == 1))
            Application.Exit();
        else
            MessageBox.Show(invalidInput1 + " "+invalidInput2 +" "+ invalidInput3);
    }
    private void textBox1_Validating(object sender, CancelEventArgs e)
    { if (textBox1.Text == "")
        {
            errorProvider1.SetError(textBox1, "You must enter a first and last name.");
            invalidInput1 = 0;
        }
        else
        {
            errorProvider1.SetError(textBox1, ""); invalidInput1 = 1;
        }
    }
    private void textBox3_KeyPress(object sender, KeyPressEventArgs e)
    { System.Text.RegularExpressions.Regex regex;
      regex = new System.Text.RegularExpressions.Regex(@"\S+@\S+\.\S+");
      if (regex.IsMatch(textBox3.Text))
      {
          errorProvider1.SetError(textBox3, ""); invalidInput3 = 0;
      }
      else
      { errorProvider1.SetError(textBox3, "Not a valid email.");
        invalidInput3 = 1;
      }
    }
    private void textBox2_Validating(object sender, CancelEventArgs e)
    { if (textBox2.Text == "")
        {
            errorProvider1.SetError(textBox2, "You must enter a first and last name.");
            invalidInput2 = 0;
        }
        else
        {
            errorProvider1.SetError(textBox2, ""); invalidInput2 = 1;
        }
    }
    private void textBox3_Validating(object sender, CancelEventArgs e)
    { if (textBox3.Text == "")
        {
            errorProvider1.SetError(textBox3, "You must enter a first and last name.");
            invalidInput3 = 0;
        }
        else
        {
            errorProvider1.SetError(textBox3, ""); invalidInput3 = 1;
        }
    }
}
}

```

Задача 3. Да се създаде приложение *Validation2* за валидиране на текстови полета (от *min double* и *int*, като се използва манипулатора на събитие *KeyPress*).

Примерно решение:

```

namespace Validation2
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
    private void textBox1_KeyPress(object sender, KeyPressEventArgs e)
        {
            if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar) &&
                (e.KeyChar != '-') && (e.KeyChar != '.'))
            {
                e.Handled = true;
            }
            // only allow one decimal point
            if ((e.KeyChar == '.') && ((sender as TextBox).Text.IndexOf('.') > -1))
            {
                if ((sender as TextBox).Text.IndexOf('.') > -1)
                    e.Handled = true;
            }
            // only one minus - first place
            if ((e.KeyChar == '-') && ((sender as TextBox).Text.IndexOf('-') > -1))
            {
                e.Handled = true;
            }
        }
    private void textBox2_KeyPress(object sender, KeyPressEventArgs e)
        {
            // only allow one integer
            if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar))
            {
                e.Handled = true;
            }
        }
    private void textBox3_KeyPress(object sender, KeyPressEventArgs e)
        {
            // only allow one integer
            for (int h = 58; h <= 127; h++)
            {
                if (e.KeyChar == h) //58 to 127 is alphabets tat will be blocked
                {
                    e.Handled = true;
                }
            }
            for (int k = 32; k <= 47; k++)
            {
                if (e.KeyChar == k) //32 to 47 are special characters tat will be blocked
                {
                    e.Handled = true;
                }
            }
        }
    }
}

```

Задача 4. Да се създаде приложение *Validation 3* за валидиране на текстови полета (от тип *int*), като се използва регулярен израз приложен върху манипулатора на събитие *TextChanged*.

Примерно решение:

```
using System.Text.RegularExpressions;
...
private void textBox1_TextChanged(object sender, EventArgs e)
{
    if (System.Text.RegularExpressions.Regex.IsMatch(textBox1.Text, "[^0-9]"))
    {
        MessageBox.Show("Please enter only numbers.");
        textBox1.Text.Remove(textBox1.Text.Length - 1);
    }
}
```

Задача 5. Да се създаде приложение *Валидиране на дата*. Датата да се въвежда в текстово поле, а валидирането да става чрез регулярен израз след кликване върху бутон.

Примерно решение:

```
using System.Text.RegularExpressions;
.....
public void Button1_Click(System.Object sender, System.EventArgs e)
{
    if (Information.IsDate(textBox1.Text))
        MessageBox.Show(textBox1.Text + " is a valid date: " + Information.IsDate(textBox1.Text));
    else
        MessageBox.Show(textBox1.Text + " is not a valid date: " + Information.IsDate(textBox1.Text));
}
```

Задача 9. Да се създаде приложение *Валидна дата*, което валидира данните въведени в текстово поле, чрез регулярен израз и след натискане на бутон.

Примерно решение:

```
using System.Text.RegularExpressions;
...
public void Button1_Click(System.Object sender, System.EventArgs e)
{
    if (Information.IsDate(textBox1.Text))
        MessageBox.Show(textBox1.Text + " is a valid date: " + Information.IsDate(textBox1.Text));
    else
        MessageBox.Show(textBox1.Text + " is not a valid date: " + Information.IsDate(textBox1.Text));
}
```

Задача 6. Да се създаде приложение *Валидиране на e-mail*. За целта да се добави текстово поле. Валидацията да се извърши при кликване на бутон, като се приложи регулярен израз.

Примерно решение:

```
using System.Text.RegularExpressions;
public void Button1_Click(System.Object sender, System.EventArgs e)
{
    string expression = default(string);
    expression = "^[0-9a-zA-Z]([-\\w]*[0-9a-zA-Z])*@([0-9a-zA-Z]([-\\w]*[0-9a-zA-Z]\\.)+[a-zA-Z]{2,9})$";
    if (Regex.IsMatch(textBox1.Text, expression))
    {
        MessageBox.Show("Valid Email address ");
    }
    else
    {
        MessageBox.Show("Not a valid Email address ");
    }
}
```

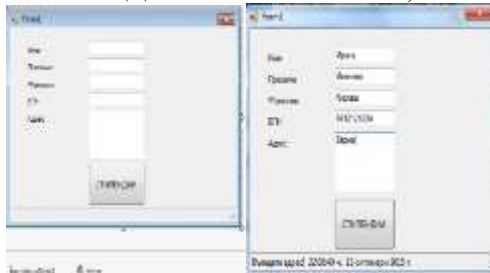
}

}

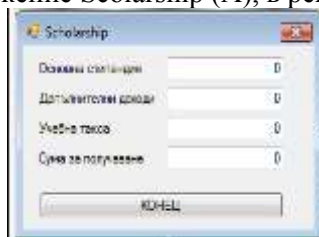
Задача 7. Да се създаде приложение *Scolars-ship* с 2 форми *Form1* и *ScolarShip*. Да се валидират текстовите полета. Във *Form1* да може да се попълва информация за студент: три имена (изисква се валидация за кирилица, имената да започват с главна буква), ЕГН (изисква се валидация за цифрово съдържание и ограничение 10 символа) и адрес на студент (валидация за кирилица).

Форма *Scolarship* да съдържа 3 числови полета за стипендия, допълнителни доходи и семестриална такса на студента. Да се валидират данните в тези полета, така, че да допускат въвеждане само на цифри и десетична точка. Сумата за получаване да се изчислява автоматично:

Резултат = (Основна стипендия + Допълнителни доходи) - Учебна такса (7)



Фиг.5.6. Интерфейс на приложение *Scolars-ship* (A), в режим проектиране и изпълнение



Фиг.5.7. Интерфейс на приложение *Scolarship* (B), в режим изпълнение

Задача 8. Да се разработи приложение *Зодиак*, което по зададена дата на раждане (чрез контрола *dateTimePicker*) да изведе зодиакалния знак в *listBox* контрола.



Фиг.5.8. Интерфейс на приложение *Зодиак*, в режим изпълнение

6.Задачи за самостоятелна работа

Задача 1. Да се създаде приложение *Работна заплата*. При стартиране да се прочита текстов файл съдържащ информация за работник с формат (име, заплата, месец, година). Информацията да се зарежда в подходящи динамични контроли. Приложението да се реализира с опция за добавяне и изтриване на запис в текстовия файл.

Задача 2. Да се разработи приложение за *MathQuiz* – тестер по математика, да се добави таймер за отчитане на оставащото време до края на теста. Генерират се 4 примера, всеки с конкретно действие (събиране, изваждане, умножение и деление), което се извършва над две произволно генерирани цели числа, в интервала от 1 до 50. При стартиране на тестера, чрез бутона „Start Game” се задейства таймер, който отброява оставащото до края време за решаване на теста.



Фиг.5.9. Интерфейс на приложение MathQuiz – тестер по математика (А), в режим проектиране



Фиг.5.10. Интерфейс на приложение MathQuiz – тестер по математика (В), в режим изпълнение



Фиг.5.11. Интерфейс на приложение MathQuiz – тестер по математика (С), в режим изпълнение

Задача 3. Да се създаде приложение Анкетна карта.

Анкетна карта да е за N на брой анкетиращи и да включва 8 въпроса. Резултатите да се обобщят с натрупване (т.е. към резултатите на предходния анкетиран да се добавят записите на следващия анкетиран) и изведат в текстов файл. Да се използва за удобство контролата tabControl и свойството и TabPages[0]:

```
tabControl1.TabPages[0].Text = "Анкета страница 1";
tabControl1.TabPages[1].Text = "страница 2";
```

Анкета стр. 1 **стр. 2**

1) Посочи 5 дисциплини, които ти харесват най-много. Започни от тази, която е водеща за теб.

Дисциплина 1	Английски език
Дисциплина 2	Web дизайн
Дисциплина 3	Офис програмиране
Дисциплина 4	Базово програмиране
Дисциплина 5	Синтез и анализ

2) Интересът към дисциплината е свързан с

☒ Бъдеща професионална реализация
☒ Придобиване на опит
☐ Личност на преподавателя
☐ Друго

3) Отбележи интересът към дисциплините

Английски език	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☒ 6
Web дизайн	☐ 1 ☐ 2 ☐ 3 ☐ 4 ☒ 5 ☐ 6
Офис програмиране	☐ 1 ☐ 2 ☐ 3 ☒ 4 ☐ 5 ☐ 6
Базово програмиране	☐ 1 ☐ 2 ☒ 3 ☐ 4 ☐ 5 ☐ 6
Синтез и анализ	☒ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐ 6

4) За упражненията използвате

☐ Личен лаптоп
☒ Предоставен от университета компютър

Continue-->

Фиг.5.12. Интерфейс на приложение Анкетна карта, в режим изпълнение – Заглавна страница

Анкета стр. 1 **стр. 2**

5) На кое упражнение сте в момента?

4

6) Посещавате ли лекции?

☒ Да
☐ Не

7) Бяха ли полезни упражненията?

☒ Да
☐ Не

8) Какво още бихте искали да научите?

беше достатъчно

Save
Clear
Summarize
Exit
<-Back

Фиг.5.13. Интерфейс на приложение Анкетна карта, в режим изпълнение – Втора страница

Задача 4. Да се създаде приложение *Здравен статус*, което включва три избора ЕГН, ЛНЧ и Булстат. След кликване върху бутон да се извежда съобщение за коректност на данните от направения избор.

Правилата за изчисляване на ЕГН, ЛНЧ и Булстат са описани както следва:

- **Правила за пресмятане на ЕГН**

Единният Граждански Номер (ЕГН) се състои от 10 цифри. Първите 6 цифри са дата на раждане (ГГММДД), следващите 3 са поредност на раждането - число от 000 до 999 и десетата цифра е контролна.

Таблица 5.2.

номер на позицията в ЕГН	1	2	3	4	5	6	7	8	9
умножение	*	*	*	*	*	*	*	*	*
тегла	2	4	8	5	10	9	7	3	6

Коректността на ЕГН се установява чрез проверка на датата, пола на лицето и контролната цифра.

1.Проверка на датата: Проверяват се денят, месецът и годината за валидна дата. За родените преди 01.01.1900 г. към месеца се прибавя числото 20. За родените след 31.12.1999 г. към месеца се прибавя числото 40.

2.Проверка на пол: Деветата цифра на ЕГН (фиг.2) е четна за мъже и нечетна за жени.

3.Алгоритъм за изчисляване на контролната цифра:

3.1.Умножават се стойностите (фиг.2) от всяка позиция със съответното тегло, посочено под номера на позицията.

3.2.Сумират се получените произведения.

3.3.Сумата по т.3.2 се дели на 11, взема се остатъкът от деленето.

3.4.Ако полученият остатък от т.3.3. е число по-малко от 10, то става контролно число. Ако е равно на 10, контролното число е 0.

За проверка на коректност могат да се приложат следните входни данни:

ЕГН: 0041012324 (01.01.2000 г.)

ЕГН: 0951012324 (10.11.2009 г.)

ЕГН: 0009091017 (09.09.1900 г.)

ЕГН: 4521010908 (01.01.1845 г.)

ЕГН: 4501010903 (01.01.1945 г.)

Пример: 6101047500 - Мъж, с дата на раждане 04.01.1961 г. Посоченото ЕГН е верно, но такова лице не съществува (Ако се засича с База от данни предоставена от ЕСГРАОН)

- **Правила за пресмятане на контролната цифра за ЛНЧ**

ЛНЧ (Личен номер на чужденец се издава на всички чужденци, които възнамеряват да пребивават продължително време в България. Издава се от МВР.

Числата с поредност

1, 2, 3, 4, 5, 6, 7, 8, 9 се умножават съответно по

21, 19, 17, 13, 11, 9, 7, 3, 1,

сумират се получените произведения и резултата се дели на 10. Остатъкът отива като последна контролна цифра.

За проверка на коректност могат да се приложат следните входни данни:

ЛНЧ: 1212121204

ЛНЧ: 1001362670

- **Пресмятане на контролна цифра за Булстат 9 знака:**

Булстат е идентификатор, какъвто има всяка фирма в България. Булстатът може да бъде 9, 10 или 13 знака. 9 знака е за всяка фирма, 10 знака е Булстат, който е еднакъв с ЕГН на търговеца и в този случай се ползва изчисление на контролна цифра за ЕГН. 13 знака се ползва за фирми с клонове. Ако фирмата има клонове, всеки клон има Булстат с 13 знака, като първите 9 са еднакви с Булстата на фирмата – майка. Фирмите с 13 знака са много малко в България.

- **Проверка на Булстат 9 знака**

Числата с поредност

1, 2, 3, 4, 5, 6, 7, 8 се умножават съответно по

1, 2, 3, 4, 5, 6, 7, 8,

получените произведения се сумират, и резултата се дели на 11. При остатък различен от 10 – контролното число е равно на остатъка, в противен случай се ползва следният алгоритъм:

Числата с поредност

1, 2, 3, 4, 5, 6, 7, 8 се умножават съответно по

3, 4, 5, 6, 7, 8, 9, 10,

получените произведения се сумират, и резултата се дели на 11. При остатък различен от 10 – контролното число е равно на остатъка, иначе контролното число е равно на 0.

За проверка на коректност могат да се приложат следните входни данни :

ЕИК: 010100102

ЕИК: 121082521

ЕИК: 131063188



Фиг.5.14. Интерфейс на приложение Анкетна карта, в изпълнение